

Entregable 1: Resumen Propio del Problema Planteado

Planteamiento del Problema:

DiplomadosOnline.com, una plataforma educativa, ha experimentado un gran crecimiento para bien y para mal con respecto al ingreso de nuevos estudiantes y docentes. Este aumento de clientes ha superado y saturado al sistema tradicional con el que se trabaja en la academia, terminando toda documentación en un archivo de Excel que termina por resaltar una serie de fallos y crisis operativa. Entre los fallos a señalar se encuentran:

- **Desorden de Datos:** Números de cédula duplicados, nombres mal escritos y asignaciones erróneas a programas equivocados; entre los estudiantes y docentes se comparten dichos problemas.
- **Falta de Persistencia:** Al depender de un solo archivo de Excel, se pueden desatar problemáticas o embrollos si en un dado caso el mismo archivo se pierde o se corrompe, lo que resta horas de trabajo y retrasa el procedimiento que se esté llevando a cabo.
- **Retraso en la Certificación:** Entre los detalles a tener en cuenta sobre la extracción de datos del promedio de un alumno, cuando este termine su periodo académico, se pasa una cantidad de tiempo considerable solo para poder enviar dichos datos al Departamento de Certificados, terminando con que el proceso se tiene que repetir con cada unos de los cientos de alumnos que podrían estar a la espera de la entrega de diplomas.

Requerimientos del MVP:

Tal y como se encarga para desarrollar el **Motor Core (Backend)** o la base de un **Sistema de Gestión Académica (SGA-DO)**, dicho caso pide ciertos requerimientos para testear la versatilidad del mismo, más allá de considerar el sistema para tres lenguajes de programación distintos (Python, Java y C++).

Sobre las funciones que debería tener el sistema, lo más básico es guardar los datos de los alumnos (cédula, nombre, correo, programa) y de los profesores (cédula, nombre, correo, especialidad, materia) en archivos de texto diferentes, como lo son **alumnos.txt** y **profesores.txt**. Posteriormente el sistema tiene de permitir ingresar una calificación del alumno tras solicitar y comprobar su cédula, y deshacer la nota ante algún error de tipeo (con **Ctrl+Z**). Luego, analizar los alumnos registrados en el archivo e insertar en una **Cola (Queue)** aquellos que cumplan con los criterios para aprobar; exportando la lista final de graduandos a un nuevo archivo llamado **certificados_pendientes.txt**. Además, se puede mostrar un reporte general en forma de lista de los profesores activos y alumnos registrados con sus estatus. En última instancia, a la hora de salir, se guarda cualquier cambio, se limpia la memoria y se cierra la aplicación sin problemas.

Plan de Acción:

1. Planificar una lógica sobre a gestión dentro del sistema, con un diagrama UML.

2. Crear un repositorio para GitHub donde guardar el progreso, además, instalar y probar las aplicaciones y herramientas necesarias para la labor.
3. Empezar a desarrollar el código para el sistema con el respectivo lenguaje a usar, creando primero algún prototipo sobre la interfaz a base de texto.
4. Comenzar a hacer la primera función de la interfaz con alumno.txt y los datos a ingresar o mostrar, replicar y adaptar la opción para profesor.txt.
5. Programar una gestión de archivos y datos guardados anteriormente, además de un sistema de verificación para los datos de los alumnos en base a la cédula para enseñar un nuevo menú respectivo a sus notas, programar una función para deshacer algo antes un fallo reciente, y finalmente limpiar datos al cerrar.
6. Establecer parámetros respecto al programa en el que esté el alumno, asegurando que se cumpla el criterio de aprobar las notas en base a cada mínimo requerido.
7. Luego, asegurarse de que aquellos usuarios que hayan aprobado se guarden en la nueva lista a programar de certificados_pendientes.txt.
8. Crear la opción del reporte general que muestre una lista detallada de los profesores y alumnos, teniendo opciones que separen y reordenen la lista a conveniencia (separar profesores de alumnos y programas).
9. Finalmente, replicar lo hecho en un código, una vez funcional, para el siguiente código; adaptando el sistema.